

Aula 7 - Modularização do código (POO IV)

Estruturando projetos Python de forma profissional



Por que organizar o código?

O Desafio

Em projetos pequenos, um único arquivo .py pode ser suficiente. Mas conforme o projeto cresce, surgem problemas:

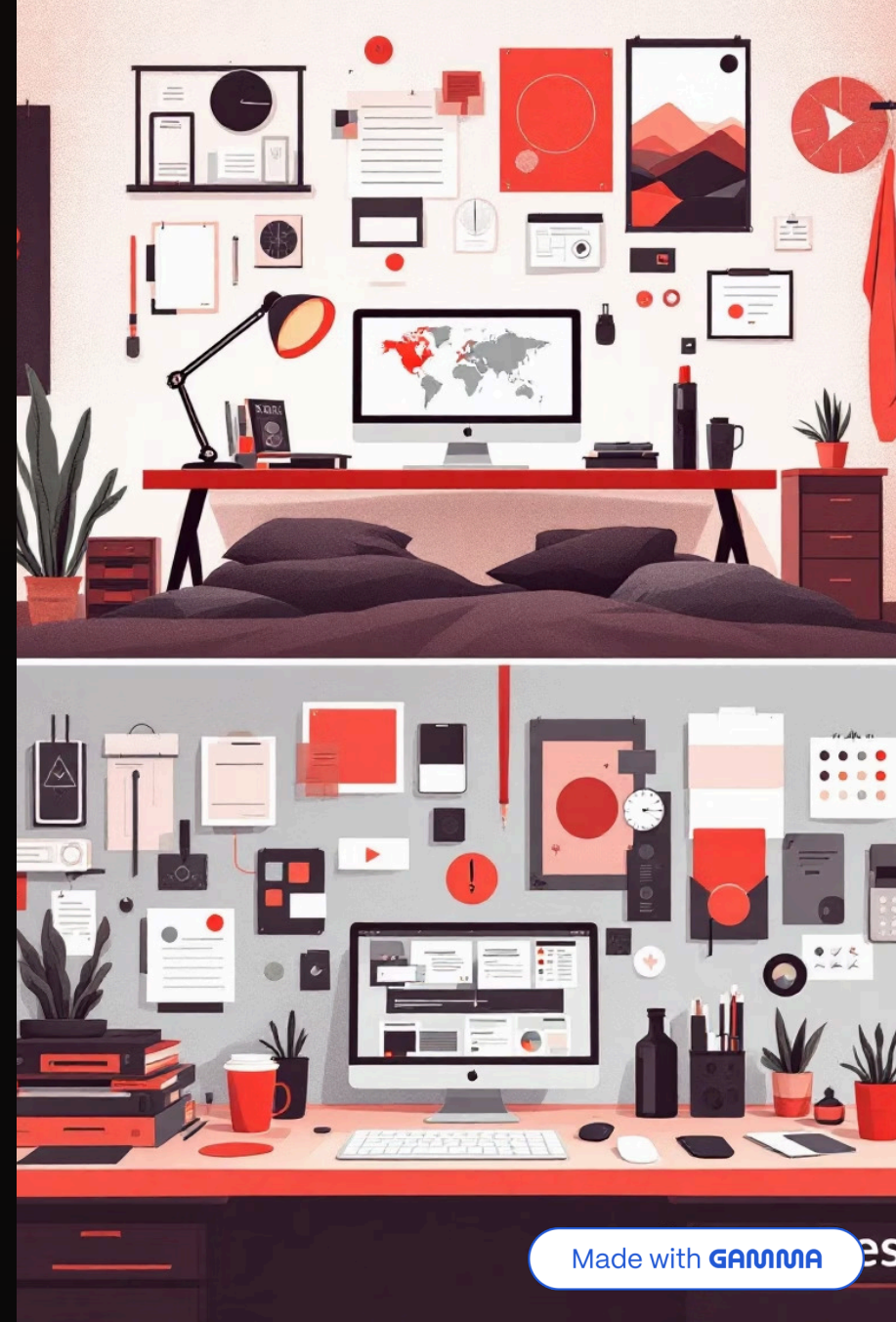
- Dificuldade para encontrar funções e classes
- Código confuso e desorganizado
- Manutenção cada vez mais complicada

Nesta aula, vamos aprender a **dividir o código em módulos e pacotes**, tornando o projeto mais limpo, escalável e fácil de manter.

A Solução

A **modularização** resolve esses problemas:

- Separa responsabilidades claramente
- Facilita testes e depuração
- Melhora significativamente a legibilidade



Estrutura básica de um projeto P00

Um projeto bem organizado segue uma hierarquia clara de pastas e arquivos. Veja como estruturar um sistema simples:



main.py

Coração do algoritmo, é aqui que executamos todo o código.



models/

Guarda as classes principais do sistema



utils/

Funções de apoio e ferramentas auxiliares

```
meu_projeto/  
|  
|—— main.py  
|—— models/  
|   |—— __init__.py  
|   |—— pessoa.py  
|   |—— aluno.py  
|—— utils/  
|   |—— __init__.py  
|   |—— funcoes_auxiliares.py
```

 **Dica:** O arquivo `__init__.py` transforma uma pasta comum em um pacote Python, permitindo a importação de módulos.

Criando um projeto passo a passo

01

Criar a pasta raiz

Crie uma pasta chamada `sistema_escolar` para abrigar todo o projeto

02

Estruturar os arquivos

Dentro da pasta, crie o arquivo `main.py` e uma pasta `models`

03

Adicionar os módulos

Na pasta `models`, crie os arquivos `__init__.py`, `aluno.py` e `professor.py`

04

Implementar as classes

Adicione o código das classes em cada arquivo específico

Exemplo de código para **aluno.py**:

```
class Aluno:
    def __init__(self, nome, matricula):
        self.nome = nome
        self.matricula = matricula

    def exibir_dados(self):
        print(f'Aluno: {self.nome} - Matrícula: {self.matricula}')
```


Acessando classes entre arquivos



Importando módulos

Para usar uma classe de outro arquivo, usamos o comando `import`. Isso mantém o código **separado e organizado**, sem misturar responsabilidades.



models/aluno.py

Define a classe Aluno



Importação

`from models.aluno import Aluno`



main.py

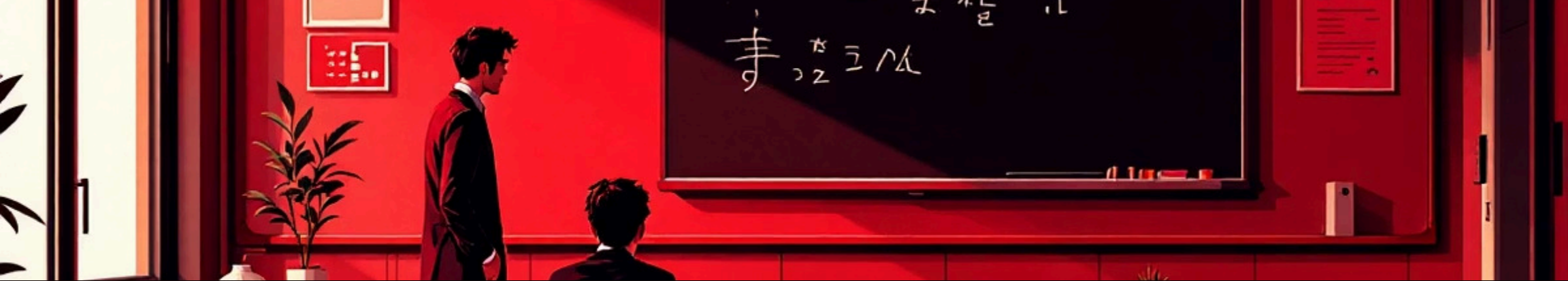
Usa a classe importada

Exemplo prático em `main.py`:

```
from models.aluno import Aluno
```

```
aluno1 = Aluno("Carlos", "A123")
```

```
aluno1.exibir_dados()
```



Exemplo com múltiplas classes

À medida que o sistema cresce, você pode criar várias classes em arquivos separados e importá-las conforme necessário.

professor.py

```
class Professor:
    def __init__(self, nome, disciplina):
        self.nome = nome
        self.disciplina = disciplina

    def apresentar(self):
        print(f"Professor {self.nome}")
        print(f"Disciplina: {self.disciplina}")
```

main.py

```
from models.aluno import Aluno
from models.professor import Professor

aluno = Aluno("João", "B456")
professor = Professor("Marcos", "Matemática")

aluno.exibir_dados()
professor.apresentar()
```

Veja como ficou simples! Cada classe tem seu próprio arquivo, e o `main.py` apenas **importa e usa** o que precisa.

Atividade 1 - Petshop

Estrutura do projeto

Monte a seguinte organização de pastas e arquivos:

```
petshop/  
├── main.py  
├── models/  
│   ├── __init__.py  
│   ├── cachorro.py  
│   └── gato.py
```

Implementação

Crie uma classe **Cachorro** e uma classe **Gato** com:

- Atributos: `nome` e `idade`
- Método `falar()` que imprima:

→ Cachorro: "Au au!"

→ Gato: "Miau!"

Teste

No arquivo `main.py`, importe ambas as classes, crie objetos e teste o método `falar()` de cada um.

📋 **Nível:** Fácil • **Tempo estimado:** 15 minutos

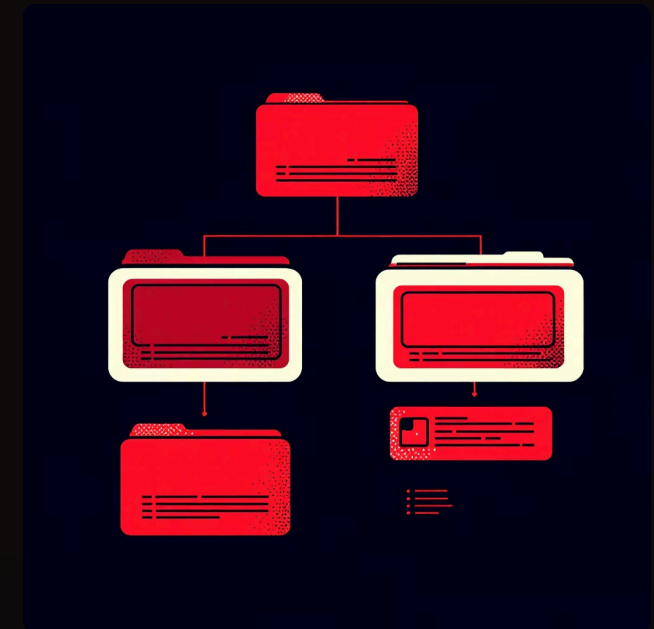


Pacotes e Submódulos

O que são pacotes?

Um **pacote** é uma pasta que contém um arquivo `__init__.py`. Ele serve para **agrupar módulos relacionados** de forma lógica.

É possível criar **submódulos** dentro de pacotes maiores, criando uma hierarquia organizada.



Exemplo de estrutura complexa

```
empresa/  
├── main.py  
├── departamentos/  
│   ├── __init__.py  
│   ├── rh/  
│   │   ├── __init__.py  
│   │   └── funcionario.py  
│   └── vendas/  
│       ├── __init__.py  
│       └── vendedor.py
```

Essa estrutura hierárquica é comum em **sistemas reais de grande porte**, onde a organização é fundamental para manter o código escalável e compreensível.

Atividade 2 - Sistema de Biblioteca

Crie um sistema para gerenciar uma **biblioteca**, aplicando os conceitos de modularização aprendidos.

Estrutura

```
biblioteca/  
├── main.py  
├── models/  
│   ├── __init__.py  
│   ├── livro.py  
│   └── usuario.py
```

Classe Livro

Atributos:

- titulo
- autor

Classe Usuario

Atributos:

- nome

Método:

- emprestar_livro(livro)

Teste

No `main.py`, crie:

- Um objeto Livro
- Um objeto Usuario
- Teste o empréstimo

 **Nível:** Fácil/Intermediário • **Tempo estimado:** 20–25 minutos



Conclusão



Organização

Separar classes, funções e utilitários em módulos e pacotes é **essencial** para projetos grandes e profissionais.



Reuso de código

Módulos bem estruturados permitem **reutilizar código** em diferentes partes do projeto.



Escalabilidade

Uma boa arquitetura permite que o sistema cresça sem se tornar confuso ou difícil de manter.



Manutenção

Código modularizado é muito mais fácil de **atualizar, corrigir e melhorar** ao longo do tempo.

"A modularização é o primeiro passo para o **desenvolvimento profissional** em Python."

